

Reviewer Pack — Rank of Young Tableaux vs Decision Tree Depth for Monotone C...

Entry b3c84360a041 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 14:37:05 UTC

Rank of Young Tableaux vs Decision Tree Depth for Monotone Circuits

Entry ID: b3c84360a041 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 14:37:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Young Tableaux) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘For every monotone circuit C with n inputs, the rank of its corresponding Young tableaux Y is bounded by the logarithm of its decision tree depth, i.e., $\log(2^{|T(C)|}) \leq \rho(Y)$, where $|T(C)|$ is the number of leaves in the decision tree for C .’, ‘For all monotone circuits C with n inputs and rank $\rho(Y) \leq k$, there exists a Young tableaux Y such that Y corresponds to C .’, ‘The mapping from a monotone circuit to its corresponding Young tableaux preserves the structure of the circuit’s computation.’]

Rationale (proposer’s reasoning):

[‘Young tableaux are a combinatorial object that can encode the structure of partitions, which are known to be related to the representation theory of Lie algebras. Monotone circuits capture the complexity of computing monotonic functions, which is relevant to computational complexity theory.’, ‘The conjecture aims to connect the representation theory through Young tableaux with the complexity of computation via circuit depth, potentially revealing a new perspective on the nature of computational complexity.’, ‘If proven, this conjecture would provide an algebraic combinatorial tool for understanding monotone circuits and could lead to more efficient algorithms for computing monotonic functions.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f9d4cb2f9249bdd3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all monotone circuits C with n inputs, the ratio of the rank $\rho(Y)$ of its corresponding Young tableaux to the logarithm of its decision tree depth $\log(2^{|T(C)|})$ is less than or equal to a threshold T . This threshold T will be determined

prior to experimentation and must satisfy the condition that 'support_fraction ≥ 0.8 AND metric_mean ≤ 3 ', where support_fraction is the fraction of circuits for which the ratio is $\leq T$, and metric_mean is the average value o

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Young Tableaux" AND "monotone circuit complexity" - "decision tree depth" IN "Algebraic Combinatorics" AND monotone circuits - "rank of Young tableaux" NEQ "logarithm of decision tree depth" monotone circuits

Top relevant hits considered: - [http://arxiv.org/abs/2104.14531v4] Matroid relaxations and Kazhdan-Lusztig non-degeneracy - [http://arxiv.org/abs/2503.14756v3] SceneEval: Evaluating Semantic Coherence in Text-Conditioned 3D Indoor Scene Synthesis - [http://arxiv.org/abs/2408.02211v2] SceneMotifCoder: Example-driven Visual Program Learning for Generating 3D Object Arrangements - [http://arxiv.org/abs/nlin/0208039v1] Effects of RF Stimulus and Negative Feedback on Nonlinear Circuits - [http://arxiv.org/abs/2405.20178v2] Non-intrusive data-driven model order reduction for circuits based on Hammerstein architectures - [http://arxiv.org/abs/1409.4826v1] Efficient Uncertainty Quantification for the Periodic Steady State of Forced and Autonomous Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def generate_monotone_circuit(n):
    if n == 1:
        return [0]
    else:
        left_size = random.randint(1, n-1)
        right_size = n - left_size - 1
        left = generate_monotone_circuit(left_size)
        right = generate_monotone_circuit(right_size)
        return sorted([x for x in left] + [x + max(left) + 1 for x in right])

def rank_young_tableau(tableau):
    n = len(tableau)
    if n == 0:
        return 0
```

```

max_length = 1
current_length = 1
for i in range(1, n):
    if tableau[i] > tableau[i-1]:
        current_length += 1
    else:
        max_length = max(max_length, current_length)
        current_length = 1
return max(max_length, current_length)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    support_count = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 5 instances per size
            circuit = generate_monotone_circuit(n)
            depth = len(circuit) - 1
            tableau = [circuit.index(i) for i in sorted(circuit)]
            rank = rank_young_tableau(tableau)
            ratio = rank / (depth + 1)
            total_metric_value += ratio
            instances_tested += 1
            if ratio <= 3:
                support_count += 1

    mean_metric_value = total_metric_value / instances_tested
    support_fraction = support_count / instances_tested
    conjecture_holds = support_fraction >= 0.8 and mean_metric_value <= 3
    counterexample = "" if conjecture_holds else "support_fraction < 0.8 or metric_mean > 3"

    return {
        "metric_name": "Ratio of Rank to Decision Tree Depth",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):

```

```

    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"support_fraction < 0.8 or metric_mean > 3\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_fraction < 0.8 or metric_mean > 3")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 80, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 51, in run_trial
    circuit = generate_monotone_circuit(n)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 24, in generate_monotone_circuit
    left = generate_monotone_circuit(left_size)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 24, in generate_monotone_circuit
    left = generate_monotone_circuit(left_size)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 24, in generate_monotone_circuit
    left = generate_monotone_circuit(left_size)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 25, in generate_monotone_circuit
    right = generate_monotone_circuit(right_size)
            ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2aaf3c8.py", line 22, in generate_monotone_circuit
    left_size = random.randint(1, n-1)
                ~~~~~^~~~~~
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ~~~~~^~~~~~
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate the cause of the crash and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15181
2	preregistration	ollama_remote	glm4:latest	0	0	10318
3	novelty	ollama_remote	glm4:latest	0	0	8625
4	novelty	ollama_remote	glm4:latest	0	0	9341
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13332
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10639
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10876
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9973
9	judge	ollama_remote	glm4:latest	0	0	18009

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106295 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b3c84360a041.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b3c84360a041.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b3c84360a041.tar.gz` (if generated)